Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Permuting Matrix Columns for Improved Block-Sparse Pruning

Master Thesis

2026
Bc. Jitka Muravská

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Permuting Matrix Columns for Improved Block-Sparse Pruning

Master Thesis

Study Programme:	Computer Science
Field of Study:	Computer Science
Department:	Department of Computer Science
Supervisor:	Mgr. Vladimír Boža, PhD.

Bratislava, 2026
Bc. Jitka Muravská

20174181

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Jitka Muravská
Študijný program: informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: Permuting Matrix Columns for Improved Block-Sparse Pruning
Preusporiadanie stĺpcov matice pre efektívnejšie blokovo riedke prerezávanie

Anotácia: V súčasnosti používaných neurónových sieťach sa najviac času venuje maticovému násobeniu medzi váhami a aktiváciami. Ukazuje sa, že je možné nahradiť veľkú váhových matíc nulami bez veľkej straty presnosti. Napriek tomu sa praktické zrýchlenie nedarí dosiahnuť, keďže násobenie riedkych matíc je na bežnom hardvéri neefektívne. Bloková riedkosť (block sparsity) zoskupuje nuly do malých maticových blokov a ponúka oveľa priaznivejší hardvérový profil.

Ji a kol. [1] ukázali, že je možné maticu váh najprv permutovať, a tým dosiahnuť lepšiu presnosť pri rovnakej miere riedkosti. Cieľom tejto práce je preskúmať a vylepšiť algoritmy na permutáciu matíc.

[1] Ji, Yu, a kol. „Tetris: Tile-matching the tremendous irregular sparsity.“ Advances in neural information processing systems 31 (2018).

Vedúci: Mgr. Vladimír Boža, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Spôsob sprístupnenia elektronickej verzie práce:
bez obmedzenia

Dátum zadania: 01.12.2023

Dátum schválenia: 30.04.2026 prof. RNDr. Rastislav Kráľovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Comenius University Bratislava
Faculty of Mathematics, Physics and Informatics

THESIS ASSIGNMENT

Name and Surname:	Bc. Jitka Muravská
Study programme:	Computer Science (Single degree study, master II. deg., full time form)
Field of Study:	Computer Science
Type of Thesis:	Diploma Thesis
Language of Thesis:	English
Secondary language:	Slovak

Title: Permuting Matrix Columns for Improved Block-Sparse Pruning

Annotation: In currently used neural networks, a significant portion of computational time is dedicated to matrix multiplications involving weight matrices and activations. Research indicates the feasibility of pruning substantial portions of weight matrices to zero without a significant sacrifice in accuracy. Despite this, practical speed-ups remain elusive, as the multiplication of sparse matrices is inefficient on current hardware. Block sparsity offers zeros in small blocks of matrices and ones, and offers a much friendlier hardware profile.

Ji et al. [1] showed that it is possible to permute the weight matrix first and achieve better accuracy for the same sparsity. The goal of this thesis is to explore and improve matrix permutation algorithms.

[1] Ji, Yu, et al. "Tetris: Tile-matching the tremendous irregular sparsity." Advances in neural information processing systems 31 (2018).

Supervisor:	Mgr. Vladimír Boža, PhD.
Department:	FMFI.KAI - Department of Applied Informatics
Head of department:	doc. RNDr. Tatiana Jajcayová, PhD.
Assigned:	01.12.2023
Approved:	30.04.2026

prof. RNDr. Rastislav Kráľovič, PhD.
Guarantor of Study Programme

.....
Student

.....
Supervisor

Declaration of Honour: I declare that I have written the entire diploma thesis "Permuting Matrix Columns for Improved Block-Sparse Pruning", including all its appendices and figures, independently, using only the literature listed in the attached bibliography and with the assistance of artificial intelligence tools. I declare that the use of AI tools was in accordance with applicable legal regulations, academic rights and freedoms, ethical and moral principles, and that I have maintained academic integrity throughout the process. The following AI tools were used in the preparation of this thesis: Claude — for rewording text, assistance with LaTeX, help with implementing code for generating figures, and code formatting. The author is responsible for the correctness of the final form of the text.

Acknowledgments: I would like to thank my supervisor for his guidance and cheering me on when I thought nothing was working. For the emotional support, the gratitude goes also to my family and friends, especially to my boyfriend for his patience, understanding, and never-ending support during the stressful times of writing this thesis.

Abstrakt

Moderné transformerové modely, od veľkých jazykových modelov po vizuálne transformery (angl. Vision Transformer), dosahujú pôsobivé výsledky na úlohách, no pri inferencii vyžadujú značné výpočtové a pamäťové zdroje. Orezávanie znižuje tieto náklady tým, že časť váh nastaví na nulu, ideálne s malou alebo žiadnou stratou kvality modelu. Blokové orezávanie robí výsledný vzor riedkosti priaznivejším pre hardvér, no za cenu: zoskupovanie váh do blokov pred orezávaním môže uväzniť dôležité váhy v inak málo hodnotných blokoch. Algoritmus TETRIS [16] tento problém rieši permutovaním dimenzií matice váh pred orezávaním, pričom na nájdenie dobrých permutácií používa nenásytné lokálne vyhľadávanie. V tejto práci predstavujeme Global Assignment TETRIS (GA-TETRIS), variant, ktorý nahrádza lokálne vyhľadávanie presným riešením úlohy minimálneho váženého bipartitného párovania v každej iterácii, doplnené o injektovanie šumu a vylepšovanie pomocou náhodných výmen na únik z lokálnych optím. GA-TETRIS porovnávame s pôvodným algoritmom TETRIS a tromi ďalšími heuristickými algoritmami — Sort-by-Norm a Random-Swaps — a so základným riešením Block-Wanda bez permutácie na dvoch Vision Transformeroch (0,9M a 13,4M parametrov) a modeli SmolLM2-360M. Pri cieli orezávania na úrovni jednotlivých vrstiev dosahuje GA-TETRIS najvyššie zlepšenie skóre pri širokých blokoch naprieč všetkými troma modelmi. Pri vyhodnotení celého modelu výkon prudko klesá s rastúcou šírkou bloku pri každom modeli; iba Väčší ViT pri bloku 1×2 si zachováva podstatnú presnosť. V tomto jedinom režime je GA-TETRIS najlepším algoritmom o 1,85 percentuálneho bodu pred druhým najlepším. Na modeli SmolLM2 pri rovnakej šírke bloku však jednoduchá heuristika Sort-by-Norm produkuje najnižšiu perplexitu — hoci stále približne sedemnásobne horšiu ako neorezaný model — čo naznačuje, že najlepší algoritmus závisí od šírky bloku aj od modelu.

Kľúčové slová: blokové orezávanie, lineárne priradenie, TETRIS, permutácia

Abstract

Modern transformer models, from large language models to vision transformers, achieve impressive task performance but require substantial compute and memory at inference. Pruning reduces this cost by zeroing out a fraction of its weights, ideally at little or no loss in task quality. Block pruning makes the resulting sparsity pattern hardware-friendly, but at a cost: grouping weights into blocks before pruning can trap important weights inside otherwise-low-value blocks. The TETRIS algorithm of [16] addresses this by permuting the dimensions of the weight matrix before pruning, using a greedy local search to find good permutations. We propose Global Assignment TETRIS (GA-TETRIS), a variant that replaces the local search with an exact solution to a minimum-weight perfect matching in a bipartite graph problem at each iteration, augmented with noise injection and random-swap refinement to escape local optima. We evaluate GA-TETRIS against the Original TETRIS and three other heuristic based algorithms, the Sort-by-Norm and Random-Swaps, and the no-permutation Block-Wanda baseline on two Vision Transformers (0.9M and 13.4M parameters) and SmolLM2-360M. On the per-layer pruning objective, GA-TETRIS achieves the highest score improvement at wide block widths across all three models. On whole-model evaluation, performance collapses rapidly with growing block width on every model; only the Larger ViT at block 1×2 preserves substantial accuracy. In that single regime, GA-TETRIS is the best algorithm by 1.85 percentage points over the next best. On SmolLM2 at the same block width, however, the simple Sort-by-Norm heuristic produces the lowest perplexity — though still roughly seven times worse than the unpruned baseline — suggesting that the best algorithm depends on both block width and model.

Keywords: block pruning, linear sum assignment, TETRIS, permutation

Contents

Introduction

Modern neural networks contain hundreds of millions to hundreds of billions of parameters. Most of the computational cost of running them at inference is spent in linear layers, multiplying activations by large dense weight matrices. A natural way to reduce this cost is *pruning*: setting a fraction of the weights to zero, ideally without losing model quality. If the pruning pattern matches what hardware can skip, the savings translate directly into faster inference.

A simple type of pruning is *unstructured pruning*, which zeros out individual weights based on their importance to the model; the resulting sparse matrix is irregular and hard for hardware to accelerate. A potential solution to this problem is *block pruning*, which zeros out fixed-shape non-overlapping blocks of weights, producing a sparsity pattern that maps directly onto specialized matrix multiplication routines on modern GPUs. The trade-off is that block pruning is more destructive: by forcing groups of weights to be removed together, it sometimes removes important weights that an unstructured pruner would have kept. The wider the block, the more enhanced this effect.

The TETRIS algorithm [16] reduces this loss by applying a permutation along each dimension of the weight tensor before pruning. Reordering elements along a dimension does not change the network's output[1], but it changes which weights end up grouped into the same block, and therefore which weights get pruned. The Original TETRIS searches the space of permutations greedily, optimizing one dimension at a time and swapping pairs of indices within that dimension whenever the swap reduces the pruned mass, stopping at the first local optimum.

In this thesis we propose *Global Assignment TETRIS* (GA-TETRIS), a variant of TETRIS that replaces the greedy single-swap search with an exact solution to the underlying assignment problem. We observe that for a fixed pruning mask, finding the optimal permutation reduces to minimum-weight perfect matching in a bipartite graph, which can be solved in polynomial time using the linear assignment algorithm. Iterating this exact step — alongside noise injection and a second phase of random-swap

[1] In our implementation the permutation is used only as a mask-finding device: we permute the weights, apply the mask, and apply the inverse permutation before writing the weights back, so the model itself is never reordered. See the beginning of Chapter 2 for details.

refinement to escape local optima — forms the GA-TETRIS algorithm.

We compare GA-TETRIS against the Original TETRIS, and two other heuristics we propose (Sort-by-Norm, Random-Swaps with and without sorted initialization), and the no-permutation Block-Wanda baseline. We evaluate on three pretrained models: a Small ViT (0.9M parameters), a Larger ViT (13.4M), and SmolLM2-360M. The comparison is conducted at two levels: a per-layer view of how well each algorithm optimizes the pruning objective directly, and a whole-model view of what each algorithm does to downstream task quality.

The findings are mixed. On the per-layer objective, GA-TETRIS produces the largest score improvements at wide block widths consistently across all three models, with the algorithm differences concentrated in early transformer blocks. On whole-model task metrics, however, all algorithms collapse rapidly as block width grows: only the Larger ViT at block 1×2 retains substantial accuracy. In that single regime, GA-TETRIS is the strongest algorithm by 1.85 percentage points. On SmolLM2 at block 1×2, simple Sort-by-Norm produces the lowest perplexity, though still roughly seven times worse than the unpruned baseline. The best algorithm depends on the model.

Chapter 1

Preliminaries and Related work

In this chapter we provide a brief introduction to the topic, setting the stage for a detailed exploration of structural pruning techniques. Alongside, we present a concise overview of the related research in the field of neural network optimization through pruning, and the specific methods we will be comparing our approach to.

1.1 Neural Networks

This thesis assumes the reader possesses a foundational understanding of Neural Networks, or more precisely Artificial Neural Networks (ANNs), including their core mechanics such as backpropagation, gradient descent, and matrix multiplication. If this is not the case, the reader is directed to [14]. The pruning techniques proposed in this work operate directly on the learned weight matrices of ANNs.

1.2 Transformer Architecture

For decades, processing sequential data was dominated by recurrent architectures [2, 5, 28], which processed inputs strictly chronologically. This imposed a severe computational bottleneck. The introduction of the Transformer architecture [29] discards recurrence, replacing it with highly parallelizable attention mechanisms.

While modern Transformers are multimodal, meaning they are capable of processing, e.g., text [3], images [10], or audio waves [13], this thesis will discuss the architecture through the lens of natural language, as all of these inputs can be reduced to sequences of discrete tokens. Treating all inputs as a sequence of tokens provides a clearer framework for explaining how the model builds contextual understanding. However, neural networks cannot process raw text directly; it must first be tokenized, mapped into continuous vector representations known as embeddings, and combined with positional information so the model knows the order of tokens. Fundamentally,

the Transformer architecture operates exclusively on these sequences of vectors. For clarity of explanation, we will use the terms word, token, and their vector embeddings interchangeably. The architecture's ability to process these sequences relies on three heavily parameterized components: Self-Attention, Multi-Head Attention, and the Multi-Layer Perceptron.

Self-Attention

The core innovation of the Transformer is **Self-Attention**. Instead of processing a sentence word-by-word, the model examines the entire sequence of tokens simultaneously. It mathematically calculates how much "attention" each word should pay to every other word to derive its true contextual meaning.

The foundational equation of this mechanism is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

To understand how this operates in practice, consider the sentence: *"The bank of the river."* The word "bank" is inherently ambiguous; it could refer to a financial institution or the side of a body of water. Self-attention resolves this by allowing the token "bank" to pull contextual clues from the token "river".

This routing of information is achieved by projecting the input embeddings (which exist in a high-dimensional space, d_{model}) into three distinct, smaller representation spaces. This is done by multiplying the inputs by three learned weight matrices—W^Q, W^K, and W^V—yielding the Queries (Q), Keys (K), and Values (V):

- **The Query (Q):** Represents what the current token is "looking for." (e.g., The token "bank" might project a query vector seeking environmental or financial context).
- **The Key (K):** Represents what information a token "contains." (e.g., The token "river" projects a key vector signaling it is a body of water).
- **The Value (V):** Represents the actual semantic payload of the token that will be passed along if the token is attended to.

The complete mechanism of self-attention is visualized in Figure 1.1. The specific steps of the mechanism are explained in the following subsections.

Alignment Score (QK^T)

To determine how relevant the words are to one another, the model computes the dot product of the Query matrix and the transposed Key matrix (QK^T). This results in

an $N \times N$ matrix of raw alignment scores, where N is the input sequence length. If the Query vector for "bank" closely aligns with the Key vector for "river", their dot product will yield a massive positive number. These raw scores are then scaled down by $\sqrt{d_k}$ (the square root of the dimension of the key vectors) to maintain numerical stability.

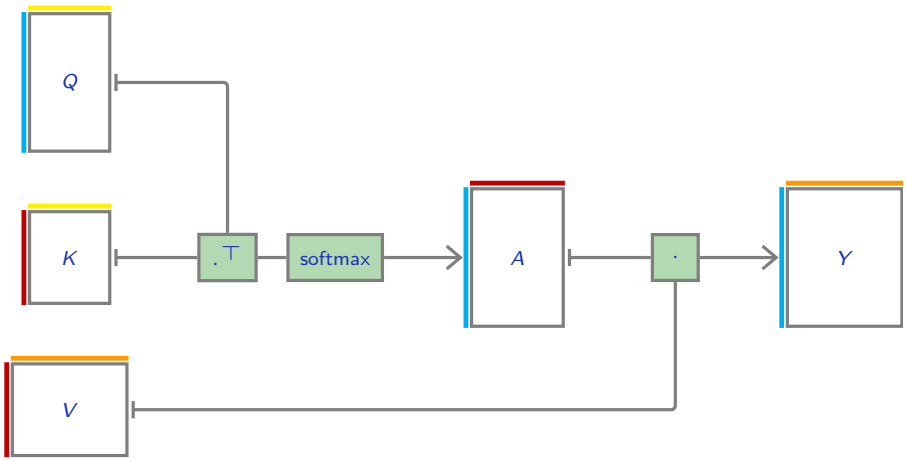

Figure 1.1: Self-attention mechanism diagram. Adapted from [12].

Softmax Transformation

Next, the scaled scores are passed through the standard softmax function to produce the final attention weight matrix, denoted as A in Figure 1.1:

$$A_{ij} = \frac{e^{Z_{ij}}}{\sum_{k=1}^{N} e^{Z_{ik}}}, \qquad \text{where } Z = \frac{QK^T}{\sqrt{d_k}}.$$

Crucially, this softmax is applied **row-wise** across the $N \times N$ matrix. For a given row i (representing the current query token), the function exponentiates the raw score it shares with every key token j, and divides it by the sum of all exponentiated scores in that row.

This guarantees that the attention weights for any single word always sum exactly to 1.0. The word "bank" now has a normalized "attention budget" to spend. It might allocate 0.65 of its attention to "river", 0.3 to "of", and 0.05 to itself.

Final Output

Finally, this row-wise probability distribution matrix (A) is multiplied by the Value matrix (V) to produce the final output sequence, Y. The resulting output vector for the word "bank" is a weighted sum: it pulls 65% of the semantic payload from

"river", fundamentally altering its own embedding to represent a "riverbank" rather than a financial institution, before passing the updated vector to the next layer of the network.

Multi-Head Attention

While a single self-attention mechanism is powerful, natural language contains many overlapping layers of syntactic and semantic complexity. In our previous example, an attention mechanism successfully linked the word "bank" to "river" to establish environmental context. However, a single attention calculation only produces one set of attention weights, which we can interpret as the model only being able to focus on one specific aspect of the language at a time.

To capture the full depth of language, the Transformer architecture employs **Multi-Head Attention**, as illustrated in Figure 1.2. Instead of using a single set of weight matrices, the architecture maintains h independent sets of learned projection matrices $(W_i^Q, W_i^K, W_i^V \text{ for } i = 1, \ldots, h)$.

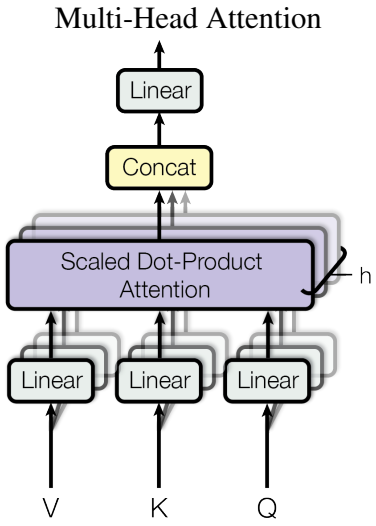

Figure 1.2: Multi-Head Attention mechanism diagram. Adapted from [29].

Crucially, each of these h "heads" observes the entire d_{model}-dimensional input embedding. The model does not physically slice the input vector into pieces; rather, each head uses its unique matrices to project the full d_{model}-dimensional embedding into a smaller space of dimension d_k for faster computation.

Thanks to the breaking of symmetry during training (e.g., through random initialization), each head naturally diverges. For instance, while one head might optimize its weights to track subject-verb agreement, another might independently learn to track emotional sentiment.

Once all h parallel attention calculations are complete, the model is left with h separate output vectors of size d_k for each token. As shown at the top of Figure 1.2, these vectors are concatenated into a single vector of size d_{model}. This concatenated vector is a collection of separate perspectives. To combine them, the vector is multiplied by a final output projection matrix (W^O) to fuse them into one highly enriched representation.

Multi-Layer Perceptron

While the attention mechanism routes information between tokens, the deep transformation of this data is performed by a Multi-Layer Perceptron (MLP) applied after the multi-head attention.

The MLP consists of two linear layers with a non-linear activation function (such as ReLU) in between. The first linear layer is a projection into an expanded higher-dimensional space, and the second linear layer is a projection compressing the representation back to the base dimension.

Language Models

Language Models (LMs), or Large Language Models (LLMs) [9, 24], are ANNs comprised of the mechanisms described above. They are designed to generate text sequentially by predicting the next word.

Structurally, an LLM is a deep, repeating stack of Multi-Head Attention and MLP layers. The purpose of the model is to generate the next word by calculating a probability distribution over the entire vocabulary given the previous context. Formally, given a sequence of tokens $x_1, x_2, \ldots, x_N$, the model is trained to learn the conditional distribution of the next token given all preceding tokens,

$$P(x_n \mid x_1, x_2, \ldots, x_{n-1}),$$

and the joint probability of the full sequence factorizes via the chain rule as

$$P(x_1, x_2, \ldots, x_N) = \prod_{n=1}^{N} P(x_n \mid x_1, \ldots, x_{n-1}).$$

This conditional distribution is produced by passing the output of the final layer through a linear projection, followed by a softmax function. At inference time, the model generates text by sampling from this distribution one token at a time.

During training, an entire sequence is fed into the model simultaneously. The model's objective is to learn $P(x_n \mid x_1, \ldots, x_{n-1})$ for every position n in the sequence. If the standard self-attention mechanism were used, the model would use the context of the correct next word to calculate its probabilities, effectively "cheating" and failing to

actually learn to predict the next word. To prevent this, the self-attention mechanism applies a mask. This mask sets the attention scores of all future tokens to negative infinity (which are then zeroed out by the softmax). This mathematically blinds the model to words it has not yet generated, forcing it to genuinely learn $P(x_n \mid x_1, \ldots, x_{n-1})$ based strictly on prior context.

Vision Transformers

Another ANN based on the transformer architecture is the Vision Transformer (ViT) [10]. While originally developed for image classification, the ViT architecture has since been adapted for a wide range of vision tasks, including object detection [19], image segmentation [17], and even video processing [23].

An input image is divided into $N \times N$ non-overlapping patches. Each patch is flattened into a 1D vector of pixel values and then mapped to a vector of dimension d_{model} by a learned linear layer. These patch vectors play the same role as the token embeddings in language models. The Transformer blocks themselves — multi-head self-attention followed by a multi-layer perceptron — have the same structure in both architectures. The full ViT pipeline is illustrated in Figure 1.3.

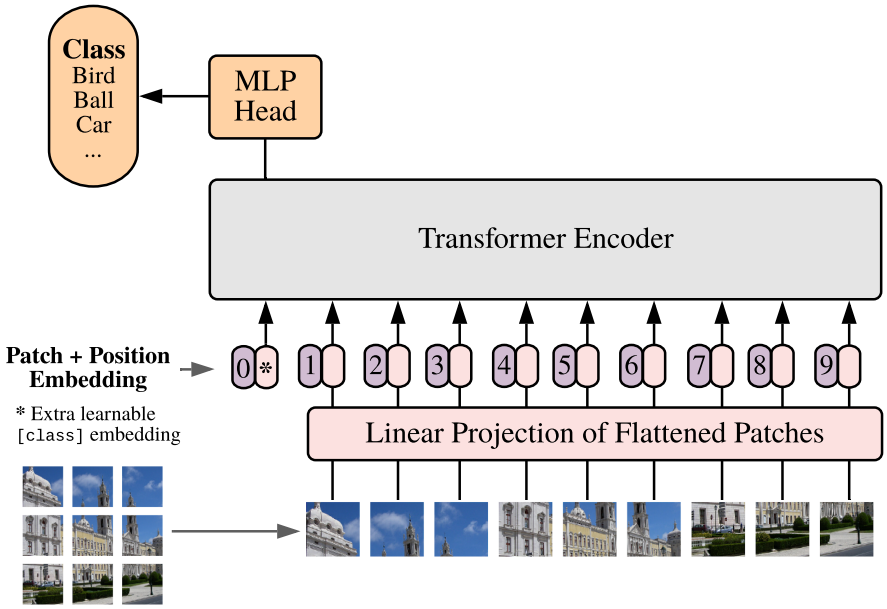

Figure 1.3: The Vision Transformer pipeline. Adapted from [10].

The key difference is in how attention is configured. Language models use a causal mask in the self-attention, which restricts each token to attending only to itself and earlier tokens. ViT uses unmasked, bidirectional self-attention — each patch attends to every other patch — since there is no notion of past or future in an image. The sequence

length is also fixed by the image resolution and patch size rather than determined by the input text.

For classification, the original ViT paper [10] prepends a learned token, called the `[CLS]` token, to the patch sequence; through self-attention this token aggregates information from all patches, and the corresponding output vector is passed through a small MLP that maps it to a probability distribution over the class labels. Some later variants, including one we use in this thesis [7, 10, 31], replace the `[CLS]` token with a global average pooling over all patch output vectors.

1.3 Pruning

The previous sections detailed the complex mechanisms of the Transformer architecture. However, beneath these high-level concepts, the majority of the computational load is performed in linear layers executing simple matrix multiplications between a batch of inputs and a learned weight matrix:

$$f(X) = XW$$

Pruning [15] is a widely adopted optimization technique designed to identify and remove redundant or less important weights from the matrix W (typically by setting their values to zero). By forcing these dense matrices to become sparse, pruning aims to significantly reduce both the computational power needed for inference and the memory required to store the model, ideally without suffering a catastrophic loss in the model's predictive accuracy.

Mathematically, pruning can be expressed as:

$$W' = W \odot M$$

where W' is the pruned weight matrix, M is a binary mask matrix of the exact same dimensions as W, and $\odot$ denotes element-wise multiplication (Hadamard product).

Mask

The core challenge of any pruning algorithm is determining the optimal mask M. In standard unstructured pruning, this is achieved by evaluating specific importance metric for each individual weight and removing weights with smallest importance. The simplest example of this metric is the absolute magnitude of the weight. The algorithm calculates an importance score, S_{ij}, for each element and identifies a scalar threshold, τ, which is typically determined by the desired target sparsity of the network. Elements with an importance score below this threshold are assumed to contribute the least to the network's output and are pruned.

Mathematically, the elements of the mask matrix, M_{ij}, are calculated as a piecewise function based on the importance score S_{ij}:

$$M_{ij} = \begin{cases} 1 & \text{if } S_{ij} \geq \tau \\ 0 & \text{if } S_{ij} < \tau \end{cases}$$

While this element-wise approach effectively preserves model accuracy, it produces an irregular, scattered distribution of zeros. This unstructured sparsity introduces severe hardware inefficiencies, directly motivating the need for structural constraints like block pruning.

Block Pruning

To resolve the hardware inefficiencies of unstructured pruning, researchers employ structured sparsity, most notably **block pruning** [22]. Instead of evaluating weights individually, the weight matrix W is partitioned into fixed-size, non-overlapping blocks (e.g., 1×4 or 4×4 sub-matrices). A particularly important special case is *slice blocks* of shape $1 \times k$, directly supported in hardware by modern accelerators [21].

The pruning criterion is then applied to the entire block collectively. The algorithm calculates an overall importance score for the block (the simplest example being the L_2 norm of the block's weights). If the overall importance of the block falls below the threshold τ, the entire block is pruned. To visualize this, an example of a block-level mask matrix M being applied to a weight matrix W to produce a structured pruned matrix W' is shown in Figure 1.4.

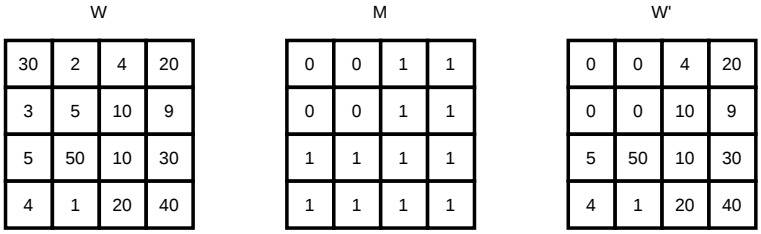

Figure 1.4: The pruned matrix W' is the result of the element-wise multiplication of the binary mask matrix M and the original weight matrix W.

Wanda Pruning

LLMs exhibit massive, emergent outlier features in their input activations [8]. Therefore, calculating the importance score based solely on the weight values is insufficient. Pruning a small-magnitude weight that multiplies an large activation feature during inference can cause catastrophic errors in the network's output.

To address this, Wanda [27] is a pruning method that calculates the importance score of each weight by considering both the weight value and the norm of the corresponding input activation feature. Specifically, the importance score is calculated as:

$$S_{ij} = |W_{ij}| \cdot \|X_j\|$$

where $\|X_j\|$ represents a chosen norm (typically the L_2 norm) of the j-th feature across the input activations.

A second contribution of Wanda is the choice of *comparison group*: rather than ranking scores globally across the layer, Wanda compares scores within each row of S independently and prunes a fixed fraction of the weights per row. This row-wise (per-output) sparsity is found to be substantially more effective for LLM pruning than layer-wise alternatives [27].

In our implementation we use the equivalent squared form $S_{ij} = (|W_{ij}| \cdot \|X_j\|)^2$, with $\|X_j\|$ calculated by accumulating the per-batch mean of squared activations across calibration batches. Both modifications preserve the per-row ranking that determines Wanda's pruning mask: squaring is monotonically increasing on non-negative values, and the per-batch mean accumulation introduces only a uniform positive scaling across all entries. In other words, sorting the scores by magnitude would produces the same order before and after the modifications.

1.4 TETRIS

Applying coarse-grained block pruning based on an unstructured score matrix S introduces the "trapped weight" problem: highly important elements ($S_{ij} \geq \tau$) grouped with near-zero scores are erroneously pruned if the block's aggregate score falls below τ_{block}.

To address this problem, TETRIS algorithm [16] permutes the dimensions of the score matrix S (and consequently the weight matrix W) before pruning to group elements with high importance scores together. Illustration of these permutations is shown in figure 1.5.

Mathematical Equivalence

TETRIS introduces two permutation vectors, α and β, for the rows and columns of W. While optimal permutations are derived from the score matrix S, they must ultimately be applied to the actual weight matrix W. To preserve mathematical equivalence, adjacent layers are adjusted accordingly.

Let $W[\alpha; \beta]$ denote the correspondingly reordered matrix, and I denote the identity permutation. Given the linear layer computation $Y = XW$, the permuted layer

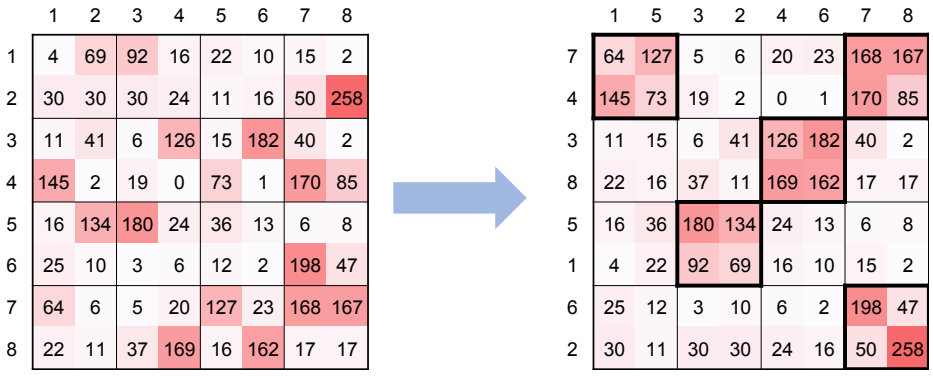

Figure 1.5: The permutations applied by TETRIS to group important elements together. Adapted from [16].

computation is expressed by:

$$Y[I;\beta] = X[I;\alpha]W[\alpha;\beta]$$

This demonstrates that reordering the rows of W by α requires the columns of the input matrix X to also be permuted by α. Likewise, reordering the columns of W by β results in an output matrix Y whose columns are permuted by β.

Assuming an activation function σ, the output passed to the next layer is $\sigma(Y[I;\beta])$. Consequently, the subsequent layer's weight matrix W_{next} must have its rows reordered by β to correctly process this permuted input. Therefore, the next layer's computation is:

$$Y_{next} = \sigma(Y[I;\beta])W_{next}[\beta;I]$$

Permutation Search

The authors generalize the number of dimensions to d. Because the possible permutations represent a massive search space, TETRIS employs a greedy search algorithm that optimizes each dimension separately and iteratively. It finds d permutations, $\alpha_1, \alpha_2, \ldots, \alpha_d$, one for each dimension. Fixing a specific dimension D, the algorithm starts from the identity permutation and iteratively swaps two indices to optimize that dimension, minimizing the objective function:

$$\arg\min_{\alpha_D} \|W[\alpha_1, \ldots, \alpha_D] \odot M\|$$

Without the loss of generality, assuming the L_1 norm is used as the pruning objective, the algorithm utilizes the importance score tensor S (e.g., derived from Wanda) and the binary mask tensor M. To evaluate swaps along the fixed dimension D, the

score tensor S is contracted with M along all dimensions except D to produce a 2D square matrix C:

$$C_{ij} = \sum_{k_1,\ldots,k_{D-1},k_{D+1},\ldots,k_d} |S_{k_1,\ldots,k_{D-1},i,k_{D+1},\ldots,k_d}|\big(1 - M_{k_1,\ldots,k_{D-1},j,k_{D+1},\ldots,k_d}\big)$$

The element C_{ij} represents the total L_1 mass of the entries that would be pruned if the i-th slice of the score tensor S were placed under the pruning pattern of the j-th slice of M (recall from Section 1.3 that $M = 1$ marks kept entries, so $1 - M$ marks pruned ones).

The gain G_{ij}, which represents the precise decrease in the objective function obtained by swapping slice i and slice j, is calculated as the difference between their current pruned mass and their pruned mass after the swap:

$$G_{ij} = C_{ii} + C_{jj} - C_{ij} - C_{ji}$$

This can also be expressed in matrix format:

$$G = L + L^T - C - C^T$$

where L is a matrix formed by broadcasting the diagonal vector of C.

To optimize dimension D, the algorithm identifies the maximum positive element G_{ij} in G and swaps indices i and j in α_D. This greedy swap is repeated iteratively until convergence, which occurs when no swap can further decrease the pruning error. The algorithm then proceeds to the next dimension until all d permutations are resolved.

Chapter 2

Methods

In this chapter we describe the methods we implement and compare in this thesis. All of them are block pruning methods (Section 1.3) that permute a dimension of the score matrix to reduce the aggregated score of the pruned elements, and differ only in how this permutation is found.

In Chapter 1 we described TETRIS in its full generality as an algorithm over a d-dimensional weight tensor with d separate permutations $\alpha_1, \ldots, \alpha_d$. For the remainder of this thesis we specialize to the 2D matrix case with slice blocks (Section 1.3), where k is assumed to divide the number of columns of the score matrix. Under this specialization only the column permutation is relevant; we denote it simply by π.

We use the column permutation purely as a mask-finding device. For each layer we (i) run a permutation algorithm to obtain a column permutation π and the block mask M computed on the permuted score matrix $S[:, \pi]$, (ii) permute the weight matrix to $W[:, \pi]$, (iii) zero out the masked entries to obtain $W[:, \pi] \odot M$, and (iv) apply the inverse permutation π^{-1} to bring the columns back to their original order:

$$W' = \big(W[:, \pi] \odot M\big)[:, \pi^{-1}].$$

The model is never reordered, so no compensation is needed in adjacent layers and the residual stream is untouched. The cost of this choice is that the resulting set of zeros is no longer aligned to consecutive $1 \times k$ slices in the original column ordering. We therefore make no claim about inference-time speedups from hardware-aligned block sparsity; we measure only how well each algorithm optimizes the per-layer pruning objective and what each algorithm does to downstream task quality, both of which depend solely on *which* weights are zeroed, not on the layout of the zeros.

All methods take as input a score matrix S (for instance, the entrywise absolute values of the weight matrix $|W|$, or Wanda scores $S_{ij} = (|W_{ij}| \cdot \|X_j\|)^2$) and produce a column permutation π that reduces the entrywise L_1 norm of the pruned submatrix. Following Wanda (Section 1.3), pruning is applied independently per row: each row is pruned to the same target sparsity, so the pruned set is determined row by row from

the current block mask. The objective is then

$$\pi^* = \arg\min_{\pi} \sum_{(i,j)\in\text{pruned}} \left|S_{i,\pi(j)}\right|,$$

where pruned collects all (i, j) pairs whose column position j is pruned in row i. In typical use cases — including all importance metrics considered in this thesis — S is non-negative by construction, so the absolute value becomes a formality and the objective reduces to a plain sum of pruned scores.

2.1 Original TETRIS

We implemented the TETRIS algorithm described in Section 1.4 adapted to the 2D matrix case with $1 \times k$ slice blocks outlined in the introduction of this chapter. Since only the column permutation is relevant, the d-dimensional contraction from Section 1.4 collapses to a single step. Given the current permuted score matrix $S[:, \pi]$ and a fixed block mask M, we form

$$C_{ij} = \sum_{k} \left|S_{k,\pi(i)}\right| \cdot \left(1 - M_{k,j}\right),$$

where C_{ij} is the L_1 mass that column $\pi(i)$ would contribute to the pruned set if placed at position j under the current mask.

$$G = L + L^\top - C - C^\top,$$

where L is the diagonal of C broadcast to a full matrix. The entry G_{ij} quantifies the decrease in pruned L_1 mass obtained by swapping columns at positions i and j in the current permutation.

The algorithm has two nested loops and is illustrated in 2.1. The *inner loop* performs greedy single-swap search at a *fixed* mask: at each step it finds the pair (i, j) with the largest gain G_{ij}, swaps π_i and π_j, recomputes G to reflect the new arrangement, and repeats until no improving swap exists ($\max_{i,j} G_{ij} \leq 0$). The mask M is not recomputed during the inner loop, so the inner loop converges to a local optimum of the fixed-mask objective under pairwise swaps.

The *outer loop* repeats this on an updated mask. After the inner loop converges and the resulting permutation is applied, the mask is recomputed from the new column arrangement, the inner loop is run again, and so on for a fixed number of outer iterations.

Algorithm 2.1: Original TETRIS (2D score matrix with column permutation specialization)

Input: Score matrix $S \in \mathbb{R}^{m \times n}$, sparsity s, block shape $(1, k)$, number of outer iterations T
Output: Column permutation π

$\pi \leftarrow (1, 2, \ldots, n)$
For $t = 1, \ldots, T$
  $M \leftarrow \text{block\_mask}(S[:, \pi],\ s,\ (1, k))$
  Repeat
    $G \leftarrow \text{gain\_matrix}(S[:, \pi],\ 1 - M)$
    $(i, j) \leftarrow \arg\max_{i', j'} G_{i', j'}$
    if $G_{i,j} > 0$ **then swap** π_i **and** π_j
  Until $\max_{i,j} G_{ij} \leq 0$
Return π

A step-by-step visualization of a single inner-loop step is shown in Figure 2.1.

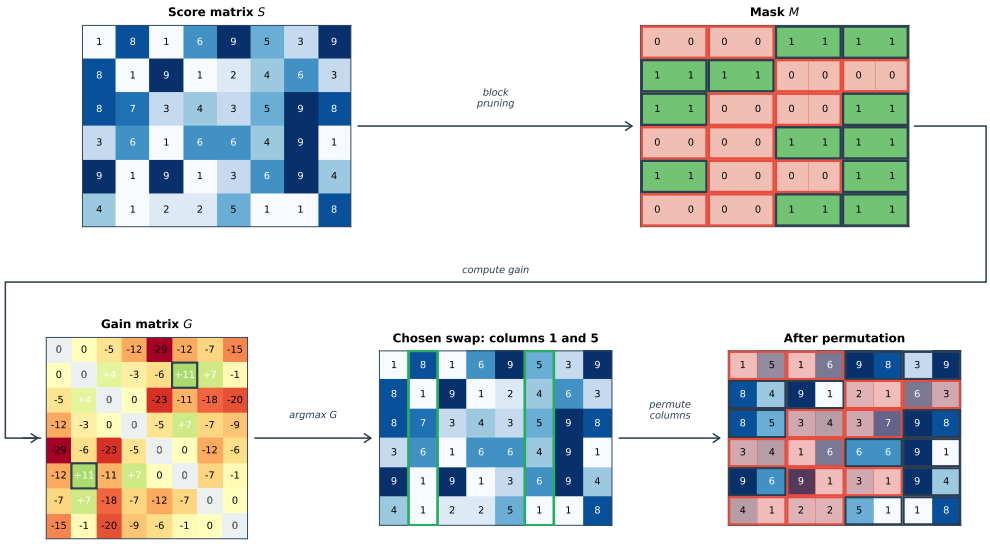

Figure 2.1: A single inner-loop step of the TETRIS algorithm: score matrix → mask → gain matrix → chosen swap → new permutation.

Because each accepted swap strictly decreases the fixed-mask objective and there are finitely many permutations, the inner loop is guaranteed to terminate. The downside is that it gets stuck at the first local optimum it reaches, with no way to escape toward permutations that would require a more complex rearrangement. The following section describes our method, which replaces this greedy single-swap inner loop with an exact assignment solution.

2.2 Global Assignment TETRIS

The greedy search in original TETRIS explores only local moves. Most permutations are not reachable by a single swap, and once the greedy search stops at the first local optimum, it has no way of moving on to better permutations that would require a more complex rearrangement.

In this section we show that for a *fixed* mask M, finding the optimal column permutation can be reformulated as a classic combinatorial optimization problem — the minimum-weight perfect matching in a bipartite graph — which can be solved exactly in polynomial time. We refer to this variant as *Global Assignment TETRIS*, abbreviated *GA-TETRIS*, reflecting the algorithmic substitution at its core: the local single-swap is replaced by a global reassignment of all columns at once.

Permutations as bipartite matchings

Any permutation of n elements can be represented as a perfect matching in a complete bipartite graph $K_{n,n}$. Let $A = \{a_1, \ldots, a_n\}$ and $B = \{b_1, \ldots, b_n\}$ be the two vertex sets, each indexed by column positions. A permutation π is encoded by the set of edges $\{(a_i, b_{\pi(i)}) : i = 1, \ldots, n\}$: the edge (a_i, b_j) is included whenever $\pi(i) = j$, i.e. when the column originally at position i is moved to position j. Every permutation yields a perfect matching, and every perfect matching in $K_{n,n}$ encodes a unique permutation.

Figure 2.2 shows the perfect matching in the bipartite graph for the permutation $[2, 4, 1, 3, 5]$.

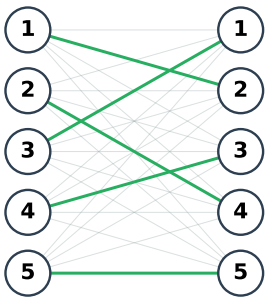

Figure 2.2: The bipartite graph $K_{5,5}$ with the perfect matching corresponding to the permutation $[2, 4, 1, 3, 5]$ highlighted.

From permutation objective to matching weight

To turn this representation into an optimization problem, we need edge weights whose sum along a matching equals the quantity we want to optimize. Given the current mask M, consider placing an original column c at position p: this placement contributes to

the pruned L_1 mass the sum of $|S_{i,c}|$ over all rows i where the mask at position p is pruned. We collect these contributions into a cost matrix:

$$C_{c,p} = \sum_{i=1}^{m} \big|S_{i,c}\big| \cdot \big(1 - M_{i,p}\big),$$

where $C_{c,p}$ is the pruned L_1 mass that column c would contribute if placed at position p under the current mask. We assign the value $C_{c,p}$ as the weight of edge (a_c, b_p) in the bipartite graph.

Under this assignment, the total weight of a perfect matching is exactly the pruned L_1 mass of the matrix after applying the corresponding permutation π:

$$\text{pruned}\big(S_{:,\pi}\big) \;=\; \sum_{c=1}^{n} C_{c,\pi(c)}.$$

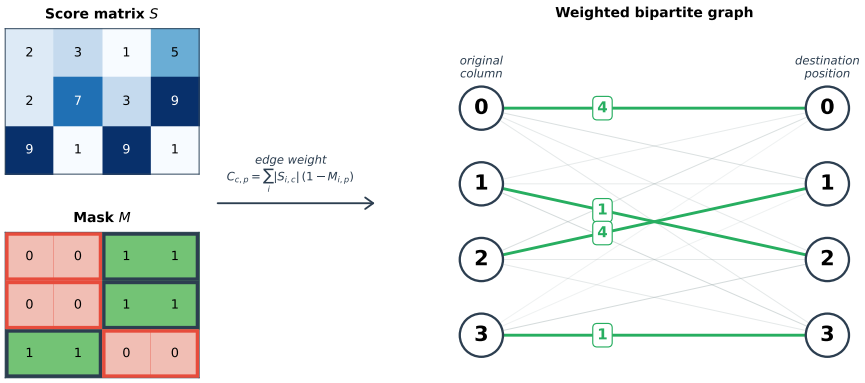

Figure 2.3: The green edges form the minimum-weight perfect matching, which corresponds to the optimal permutation for the given score and mask matrices.

Solving the minimum-weight matching

Finding the minimum-weight perfect matching in a bipartite graph is a classic problem in combinatorial optimization; it is equivalent to the *linear assignment problem* on the cost matrix C, which is solved by the Hungarian algorithm [18], with an $O(n^3)$ refinement given later by Edmonds and Karp [11]. We use the implementation provided by `scipy.optimize.linear_sum_assignment` [6, 30] that also runs in $O(n^3)$ time.

One important detail: the mask itself depends on how the columns are arranged, so solving the minimum-weight matching once only gives us the best permutation for the current mask, not the best overall. To address this, we reuse the iterative structure of the original TETRIS. In each iteration, we recompute the mask from the current permuted matrix, solve the matching problem against this updated mask, and apply the resulting permutation. The loop then continues for a fixed number of iterations.

As shown in Figure 2.4, the key difference from the original TETRIS iteration in Figure 2.1 is the use of a cost matrix and a global reassignment of all columns at once.

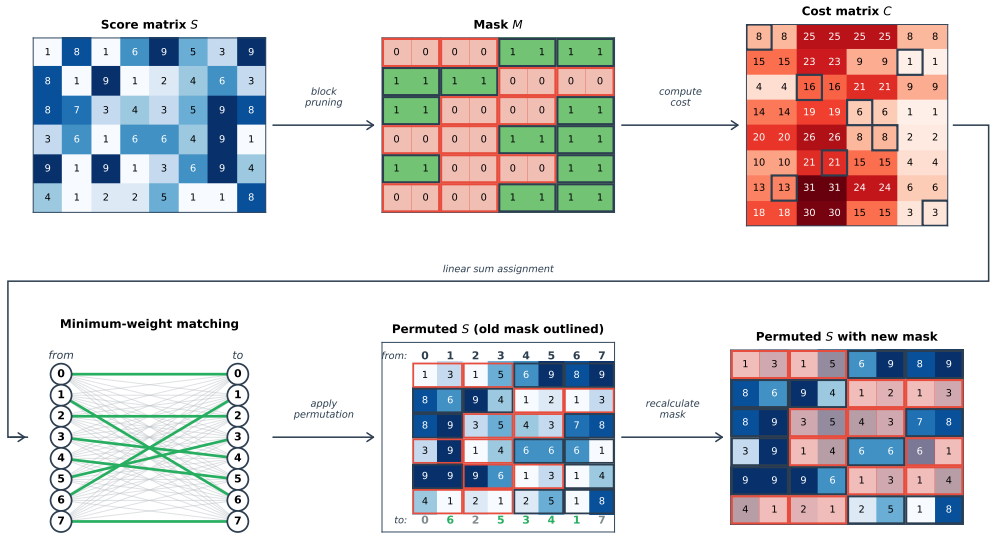

Figure 2.4: One phase-1 iteration of GA-TETRIS (with noise disabled), followed by the beginning of the next iteration

Escaping local minima

Recomputing the mask between iterations helps the algorithm explore beyond a single fixed-mask optimum, but each step still only moves to the optimum of a slightly different mask — a small jump that cannot reach permutations requiring a more complex rearrangement. We address this with two heuristics that introduce controlled randomness into the search: *noise injection* during the matching step, and *random swaps with refinement* as a second optimization phase.

Noise injection. The first heuristic is motivated by the idea of computing not the optimal permutation but the *k-th best permutation* for some moderate k. This would let the algorithm escape a local optimum by deliberately taking a slightly worse step. However, computing the k-th best permutation directly is computationally expensive for general k [4], and many of the top-ranked permutations are essentially equivalent under the block-pruning mask[1], so k would need to be quite large.

We achieve a similar effect more cheaply by perturbing the score matrix with multiplicative noise:

$$S_{\text{noisy}} = S \odot \big(\mathbf{1} + \eta \cdot Z\big), \qquad Z \in \mathbb{R}^{m \times n}, \quad Z_{i,j} \sim \mathcal{N}(0,1) \text{ i.i.d.},$$

where $\odot$ denotes element-wise multiplication, $\mathbf{1}$ is the all-ones matrix of the same shape as S, and η is the noise scale. We solve the assignment problem on S_{noisy}

[1]Within each $1 \times k$ block, all column positions share the same row-wise mask values, so swaps within a block leave the cost unchanged. The first many "best" permutations therefore differ only in such intra-block reorderings.

instead of S. The resulting permutation is exact for the perturbed problem and, with high probability, suboptimal but reasonable for the original problem — effectively a randomized neighbour of the true optimum. The noise scale decreases linearly across iterations:

$$\eta_t = \eta_0 \cdot \left(1 - \frac{t}{T_{\text{phase1}}}\right),$$

where η_0 is the initial noise scale and t indexes the outer iteration. The final iteration uses $\eta_{T_{\text{phase1}}} = 0$, recovering the exact optimal permutation under the unperturbed S.

Random swaps with refinement. The second heuristic runs after the T_{phase1} iterations of phase 1 have completed. Each of the T_{phase2} phase-2 iterations performs N_{perturb} random column swaps to displace the current permutation, then N_{refine} rounds of mask recomputation and linear sum assignment to re-optimize the resulting arrangement. If the new pruned L_1 mass is lower than the previous best by at least a tolerance ε, the new arrangement is accepted; otherwise the column arrangement is rolled back to the start of the iteration.

Algorithm summary. The full GA-TETRIS algorithm consists of the two phases above, with hyperparameters η_0, T_{phase1}, T_{phase2}, N_{perturb}, N_{refine}, and the acceptance tolerance ε. Pseudocode is given in Algorithm 2.2.

Algorithm 2.2: GA-TETRIS

```
Input:  Score matrix S ∈ ℝ^{m×n}, sparsity s, block shape (1, k),
        initial noise η_0, outer iterations T_phase1, T_phase2,
        random swaps per iteration N_perturb,
        Linear sum assignment rounds N_refine, acceptance tolerance ε
Output: Column permutation π

π ← (1, 2, ..., n)

# Phase 1: noise-injected exact assignment
For t = 1, ..., T_phase1
    M ← block_mask(S[:, π], s, (1, k))
    η_t ← η_0 · (1 − t/T_phase1)
    Sample Z ∈ ℝ^{m×n} with Z_{i,j} ~ 𝒩(0,1) i.i.d.
    S_noisy ← S ⊙ (1 + η_t · Z)
    C_{c,p} ← Σ_i |S_noisy[i, π(c)]| · (1 − M_{i,p})
    π' ← lin_sum_assigment(C)          # min-weight assignment
    π ← π ∘ π'                         # compose permutations

```

$L_{\text{best}} \leftarrow \text{pruned\_L1}(S[:, \pi])$

`# Phase 2: random swaps with linear sum assignment refinement`
`For` $t = 1, \ldots, T_{\text{phase2}}$
    $\pi_{\text{prev}} \leftarrow \pi$
    `Repeat` N_{perturb} `times`
        `Pick` i, j `uniformly at random from` $\{1, \ldots, n\}$, $i \neq j$
        `Swap` π_i `and` π_j
    `Repeat` N_{refine} `times`
        $M \leftarrow \text{block\_mask}(S[:, \pi],\ s,\ (1, k))$
        $C_{c,p} \leftarrow \sum_i \left|S[i, \pi(c)]\right| \cdot (1 - M_{i,p})$
        $\pi' \leftarrow \text{lin\_sum\_assigment}(C)$
        $\pi \leftarrow \pi \circ \pi'$
    $L \leftarrow \text{pruned\_L1}(S[:, \pi])$
    `if` $L_{\text{best}} - L > \varepsilon$ `then`
        $L_{\text{best}} \leftarrow L$
    `else`
        $\pi \leftarrow \pi_{\text{prev}}$ `# rollback`
`Return` π

2.3 Sort by column norm

Block pruning chooses which blocks to discard based on their per-block L_1 norms (Section 1.3). With $1 \times k$ slice blocks, the block norm is simply the sum of absolute values across k adjacent columns within a row. This suggests a simple heuristic: arrange the columns so that, on average, columns with large absolute values cluster together, leaving columns with small absolute values to form the pruned blocks.

The simplest implementation of this idea summarizes each column by a single number, its L_1 norm, and arranges columns in descending order of this number. Let

$$\nu_c = \sum_{i=1}^{m} \left|S_{i,c}\right|$$

be the L_1 norm of column c. We sort the columns of S in descending order of ν_c and apply the resulting permutation. After sorting, the leftmost k columns form a single block of the largest-norm columns, the next k form the second-largest block, and so on. The block mask is then computed once on the sorted matrix and applied directly — there is no iteration. Pseudocode is given in Algorithm 2.3.

Algorithm 2.3: Sort by column norm

```
Input:  Score matrix S ∈ ℝ^{m×n}
Output: Column permutation π

For c = 1, ..., n
    ν_c ← Σ_{i=1}^{m} |S_{i,c}|
π ← argsort(ν) in descending order
Return π
```

The algorithm is non-iterative and runs in $O(mn + n \log n)$ time, dominated by the cost of computing the column norms. The algorithm uses no information about the mask at all: a column's position is determined by its total magnitude alone, regardless of how its values are distributed across rows. Despite this simplicity, sort-by-norm provides a meaningful permutation in practice as we will show in Chapter 3.

2.4 Random swaps

A different way to search for a good column permutation is to abandon any direct connection between the columns and the mask, and instead explore the permutation space stochastically. The random-swaps algorithm starts from some initial permutation, repeatedly perturbs it with a batch of random column swaps, and keeps the perturbation only if it improves the pruned L_1 mass.

Concretely, at each iteration we swap a fixed fraction ρ of the columns: we sample $\lfloor \rho \cdot n \rfloor$ disjoint pairs uniformly at random and swap each pair in the current permutation. The mask is recomputed from the new column arrangement and the resulting pruned L_1 mass is compared against the best value observed so far. If the new mass is lower by at least a tolerance ε, the perturbation is accepted; otherwise the permutation is rolled back to its previous state. This continues for a fixed number T of iterations.

Two starting points are natural: the identity permutation, in which case the algorithm explores from scratch, and the permutation produced by sort-by-column-norm (Section 2.3), which gives the search a more informed starting point. We refer to the latter variant as *sort-initialized random swaps*. Pseudocode is given in Algorithm 2.4.

Algorithm 2.4: Random swaps

```
Input:  Score matrix S ∈ ℝ^{m×n}, sparsity s, block shape (1, k),
        swap fraction ρ ∈ (0, 1], iterations T,
        acceptance tolerance ε,
        sort-initialize: boolean
Output: Column permutation π
```

```
6
7   if sort-initialize then
8       π ← sort-by-column-norm(S)            # see Algorithm 2.3
9   else
10      π ← (1, 2, ..., n)
11  M ← block_mask(S[:, π], s, (1, k))
12  L_best ← pruned_L1(S[:, π], M)
13
14  For t = 1, ..., T
15      π_prev ← π
16      Sample ⌊ρ · n⌋ disjoint pairs (i_k, j_k)
17          uniformly at random from {1, ..., n}
18      For each pair (i_k, j_k), swap π_{i_k} and π_{j_k}
19      M ← block_mask(S[:, π], s, (1, k))
20      L ← pruned_L1(S[:, π], M)
21      if L_best − L > ε then
22          L_best ← L
23      else
24          π ← π_prev                        # rollback
25  Return π
```

Unlike GA-TETRIS, this algorithm performs no exact optimization step at all — every change to the permutation comes from a random swap, and the only signal driving the search is whether each batch of swaps happens to land in a better permutation.

Chapter 3

Results

We evaluate the algorithms presented in previous chapters at two levels. First, we measure each algorithm's effect on the per-layer pruning objective. We use two metrics here: *score improvement*, the relative reduction in the entrywise L_1 mass of the pruned submatrix of the score matrix S that each algorithm directly minimizes, and the *importance-weighted reconstruction error*, the activation-weighted squared error between the original and pruned weights. Both metrics are defined formally in Section 3.1. Second, we measure the effect on performance after applying the pruning on the whole model. Here we use two main metrics: perplexity for language models and accuracy for vision models. The two views answer different questions and, as we will see, do not always agree.

3.1 Setup

Models

We evaluate on three pretrained models spanning two architectures and a varying size range, summarized in Table 3.1. The two vision models are pretrained on ImageNet-1k [26] and obtained from the timm library [7, 10, 31]; the Small ViT serves as the primary model for the per-layer comparison, while the Larger ViT provides a check that our findings transfer across model scale. The language model, SmolLM2 [1], lets us verify that the patterns observed on vision transformers also appear in LM setting on a substantially larger model.

Datasets and calibration

The Wanda score $S_{ij} = (|W_{ij}| \cdot \|X_j\|)^2$ requires per-input-channel norms $\|X_j\|$ to be estimated from a small calibration set. Following Wanda [27], we use a small calibration set to estimate $\|X_j\|$: 128 ImageNet training images for the vision models and 32

Table 3.1: Models used in this chapter. Vision Transformers fuse the QKV projection into a single layer; SmolLM2 uses separate Q, K, V projections. ViT uses a two-layer MLP; SmolLM2 uses a three-layer gated MLP.

	Small ViT	Larger ViT	SmolLM2
Parameters	0.9M	13.4M	360M
Transformer blocks	9	14	32
Prunable layers per block	4	4	7

sequences from the C4 dataset [25] for SmolLM2.

Whole-model evaluation uses the standard test sets: ImageNet validation [26] for top-1 accuracy on the vision models, and WikiText-2 [20] for perplexity on SmolLM2.

Metrics

For per-layer comparison we use two metrics that capture slightly different aspects of pruning quality.

The first is *score improvement*: the relative reduction in pruned score mass that an algorithm achieves over the unpermuted baseline. Let P_{algo} be the total L_1 score mass at the cells an algorithm prunes, and P_{base} be the same quantity under the Block-Wanda baseline[1]. Then

$$\text{score\_improvement} = 100 \cdot \frac{P_{\text{base}} - P_{\text{algo}}}{P_{\text{base}}}.$$

A value of 0% means the algorithm did no better than Block-Wanda; 100% would mean every cell pruned was scoreless. This is the quantity each algorithm is designed to maximize, making it the most direct measure of algorithmic effectiveness.

The second is *importance-weighted reconstruction error*, which measures how much information the pruning destroys. Let E_{algo} be the total importance-weighted squared error between the original and pruned weights, and E_{total} be the total importance-weighted squared mass of the original weights, where the importance weight on column j is the per-input-channel norm $\|X_j\|^2$ from the Wanda score. Then

$$\text{rel\_error} = \frac{E_{\text{algo}}}{E_{\text{total}}}.$$

Concretely, since pruning sets cells to zero, E_{algo} is the Wanda-weighted squared mass at the pruned cells, and E_{total} is the Wanda-weighted squared mass of the entire weight matrix. Unlike score improvement, rel_error compares against the unpruned weights rather than against any pruning baseline.

[1] Block-Wanda applies the block mask directly with the identity permutation $\pi = (1, \ldots, n)$.

The two metrics are closely related: both measure the importance-weighted mass at pruned positions, differing primarily in their normalization.

Whole-model metrics for each domain were introduced above.

Hardware

The vision experiments ran on a workstation with an Intel Core i7-7700K CPU (4 cores, 8 threads at 4.2 GHz), 32 GB RAM, and NVIDIA Quadro RTX 5000 GPU (16 GB VRAM), running Ubuntu 22.04. The SmolLM2 experiments ran on a workstation with an AMD Ryzen Threadripper 1920X CPU (12 cores, 24 threads), 64 GB RAM, and a NVIDIA GeForce RTX 4090 GPU (24 GB VRAM), running Ubuntu 24.04. Both machines used PyTorch 2.6.0 with CUDA 12.4. Most of the pipeline runs on the GPU, including the forward pass that produces calibration norms. The linear sum assignment step in GA-TETRIS algorithms runs on CPU via SciPy's `linear_sum_assignment`, since PyTorch does not provide a built-in GPU implementation. Absolute computation times reported in this chapter are therefore not directly comparable across the two machines.

Hyperparameters

Our algorithms have several parameters which were showed in Chapter 2. Throughout the experiments we use fixed sparsity $s = 0.5$ following Wanda [27]. The most important hyperparameters, that influence the running time of the algorithms, are the block width k, number of iteration T (respectively T_{phase1} and T_{phase2} for GA-TETRIS) and number of linear sum assignment iterations in phase 2 of GA-TETRIS N_{refine}. For these we do a sweep across a range of values to understand how the tradeoff between time and score improvement behaves.

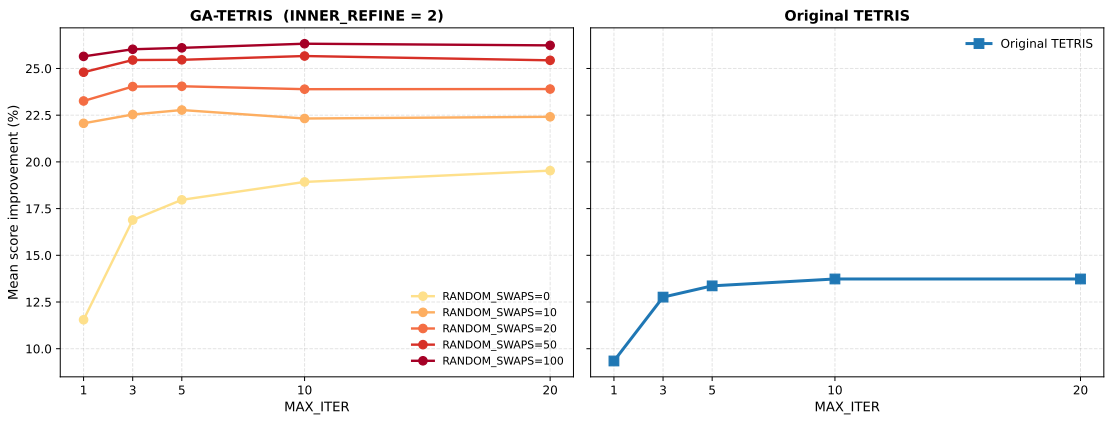

Figure 3.1: Score improvement tradeoff for Original TETRIS and GA-TETRIS as a function of iteration count. With block shape 1×8 on the Small ViT.

To choose T resp. T_{phase1}, T_{phase2} and N_{refine} we did a grid search on Small ViT for GA-TETRIS and Original TETRIS across values $1, 3, 5, 10, 20$ for T resp. T_{phase1} and just on GA TETRIS for $T_{phase2} \in \{0, 10, 20, 50, 100\}$ and $N_{refine} \in \{1, 2, 3\}$.

From the results shown in Figure 3.1, both T and T_{phase1} plateau by 10 and 5 respectively. For the number of random swaps in GA-TETRIS, we see the score improvement grows with the number of swaps but the gain past $T_{phase2} = 100$ is small relative to the cost. For GA-TETRIS to be competitive with Original TETRIS timewise we choose $T = 10$ for Original TETRIS and $T_{phase1} = 5$, $T_{phase2} = 100$, $N_{refine} = 2$ for GA-TETRIS. As shown in Table 3.2, the chosen defaults are within 0.7 percentage points of the grid maximum at significantly less compute.

For SmolLM2, where matrices are larger and $T_{phase2} = 100$ would push the full block-width sweep beyond reasonable computation time, we reduce T_{phase2} to 10; the impact of this reduction is quantified at the end of Section 3.3.

Table 3.2: Cost–quality tradeoffs around the chosen default on the Small ViT at block shape 1×8.

Configuration (T_{phase1}, T_{phase2}, N_{refine})	Score (%)	Time per layer (s)
Chosen default $(5, 100, 2)$	**26.10**	**0.379**
Grid maximum $(20, 100, 3)$	26.80	0.534
gain over default	+0.70	+41%
Lower swap budget $(5, 10, 2)$	22.78	0.046
loss vs default	−3.32	−88%
No outer iterations $(1, 100, 2)$	25.65	0.365
loss vs default	−0.45	−4%
No swap phase $(5, 0, 2)$	17.97	0.009
loss vs default	−8.13	−98%

Random-Swaps has a single tuned hyperparameter, the swap budget T_{swaps}. We tested $T_{swaps} \in \{100, 1000, 10000\}$ on the Small ViT at block width 1×8 for both the unsorted and sorted starting variants (Table 3.3). Both variants climb steadily across the full range, with no sign of convergence: $10\times$ more swaps hence $10\times$ the computation time yields 3–4 additional percentage points of score for both. We use $T_{swaps} = 1000$ throughout the rest of this chapter. Larger budgets become uncompetitive with other algorithms on the bigger models — already at $T_{swaps} = 1000$, Random-Swaps takes longer per layer than GA-TETRIS does at its chosen default.

Two additional hyperparameters were not swept in this analysis: swap fraction ρ

Table 3.3: Random-Swaps sensitivity to T_{swaps} on the Small ViT at block shape 1×8.

Variant	T_{swaps}	Score (%)	Time per layer (s)
Random-Swaps	100	10.77	0.087
Random-Swaps	1000	18.16	0.859
Random-Swaps	10000	22.78	8.580
Random-Swaps (sorted)	100	19.45	0.086
Random-Swaps (sorted)	1000	21.78	0.858
Random-Swaps (sorted)	10000	24.88	8.460

for Random-Swaps algorithms and the initial noise η_0 for GA-TETRIS's phase 1. The swap fraction was fixed at 1/30 throughout. GA-TETRIS uses multiplicative Gaussian noise during the phase 1 with the noise scale decaying linearly from 25% to 0 across the phase 1 iterations. These values were chosen empirically while developing the algorithms and not re-tuned for the comparisons in this chapter; characterizing their effects would be a useful direction for future work.

3.2 Per-layer evaluation

The most direct way to compare the algorithms is to apply each one to every prunable layer of a model and measure how well it optimizes chosen metric. This section presents that comparison on all three models across the full block-width sweep. Whether the score improvements translate into preserved model and it's performance is discussed in Section 3.3.

We can divide the compared algorithms into groups. Block-Wanda as the unpermuted baseline. Original TETRIS as baseline for sophisticated permuration finder with GA-TETRIS as our version of it to compare. Sort-by-Norm and Random-Swaps (unsorted) as simple heuristics with Random-Swaps (sorted) as improvement of just the sorting heuristic with the high cost of random swaps.

As a first look at the performance evaluation, we answer which algorithm maximizes the improvement score compared to the baseline Block-Wanda and how the comparison changes with block width.

From the heatmaps in Figure 3.2 we can see that at narrow block widths, the simple heuristics of Sort-by-Norm and Random-Swaps (sorted) perform best, while at wider blocks the TETRIS algorithms take the lead. This pattern could be explained by the fact that at narrow blocks, the combinatorial space of permutations is small enough that it is hard for the more sophisticated algorithms to find the permutation that would

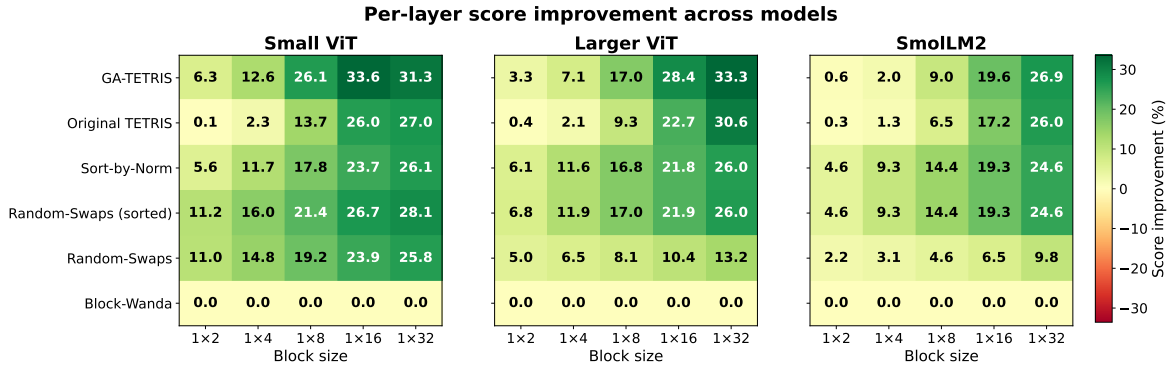

Figure 3.2: Mean (layer-wise) score improvement (%) on all models.

yield better results as the simple heuristics. While the pattern is consistent across all three models, with larger model the crossover happens at wider blocks. Except for unpermuted random swaps which perform worse with the growth of the model size. We explain this by that the combinatorial space of permutations grows with the model size, but the budget of random swaps is fixed, so the algorithm explores a smaller fraction of the space with larger models.

The fact that the sorted version improves the sorted permutation at Small ViT the most, on Larger ViT only by a little fraction $(0.7, 0.3, 0.2, 0.1, 0$ per growing block width) and on SmolLM2 not at all supports this explanation.

Table 3.4: Mean per-layer pruning time (seconds) at two block widths, across models.

Algorithm	Block	Small ViT	Larger ViT	SmolLM2
GA-TETRIS	1×2	0.251	7.645	2.527
	1×16	0.395	45.921	36.567
Original TETRIS	1×2	0.010	0.012	0.050
	1×16	0.027	0.168	1.706
Sort-by-Norm	1×2	0.001	0.002	0.002
	1×16	0.002	0.002	0.007
Random-Swaps (sorted)	1×2	0.868	2.234	7.445
	1×16	0.870	2.258	7.143
Random-Swaps	1×2	0.853	2.301	7.323
	1×16	0.858	2.281	7.170
Block-Wanda	1×2	<0.001	<0.001	<0.001
	1×16	<0.001	<0.001	0.002

With no improvement at SmolM2 and only a little improvement at the ViTs, the

computation time buffer of Random-Swaps (sorted) over Sort-by-Norm, showed in Table 3.4, is not justified, so in the next comparisons we mostly focus on selected three algorithms: Original TETRIS as fast (0.05s per layer on SmolLM2 with 1×2 blocks) simpler algorithm for finding more refined permutations, GA-TETRIS as advanced but time-costly (2.5s per layer on SmolLM2 with 1×2 blocks) version of it and Sort-by-Norm as simple very fast (0.002s per layer on SmolLM2 with 1×2 blocks) heuristic. Block-Wanda will stay as baseline for score improvement.

Figure 3.3 shows the same data as Figure 3.2 as trajectories, making three observations easier to read. First, GA-TETRIS pulls progressively ahead of Sort-by-Norm as block width grows — the gap is essentially zero at 1×4 (on SmolLM2 even negative) but reaches several percentage points by 1×16 on all three models. Second, the crossover block width at which GA-TETRIS overtakes the simple sorting heuristic is approximately the same across models (between 1×4 and 1×8 on ViT and 1×16 for SmolLM2), suggesting it is a property of the Wanda-score distribution rather than of model size. Third, on the larger models Original TETRIS catches up to GA-TETRIS at the widest block widths, narrowing the gap considerably; on the Small ViT, GA-TETRIS retains a clear lead even at 1×32.

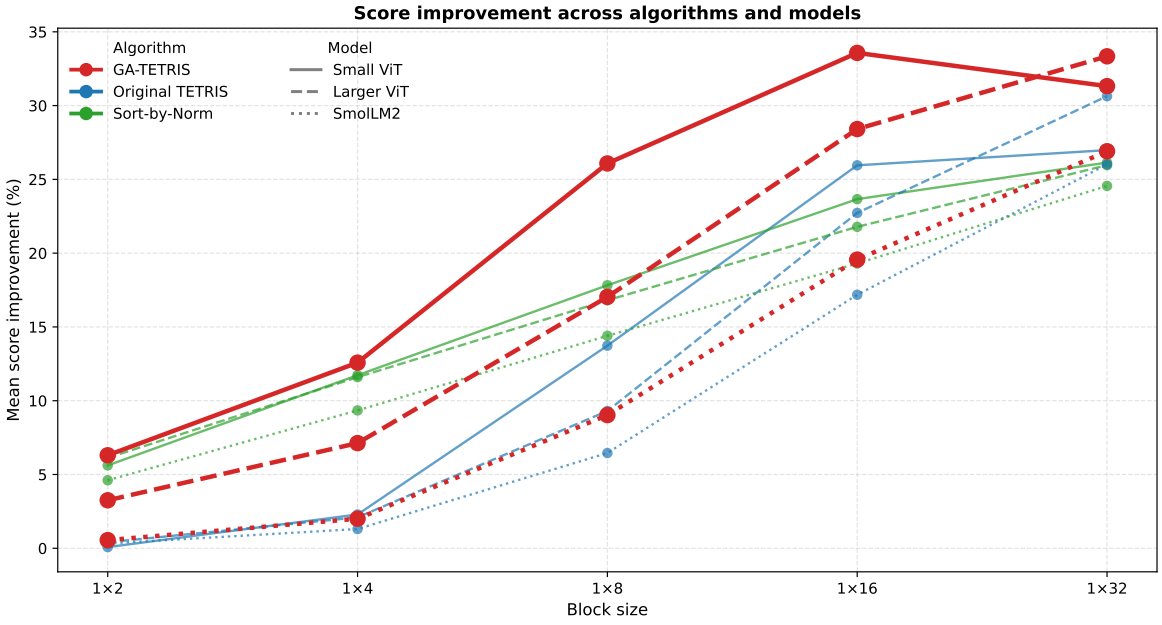

Figure 3.3: Mean (layer-wise) score improvement (%) of selected algorithms across block widths and models.

Disaggregating by block index reveals that the algorithm differences and overall score improvements are concentrated in the early blocks (Figure 3.4). We hypothesize that early blocks have different structure than later blocks, which gives the linear sum assignment step in GA-TETRIS, and especially its random-swap refinement, more room to find better permutations. By the deeper blocks, this structure is changed and

there is less left to easily gain over Block-Wanda.

The SmolLM2 panel is denser because the model has more transformer blocks, compressing the dots horizontally.

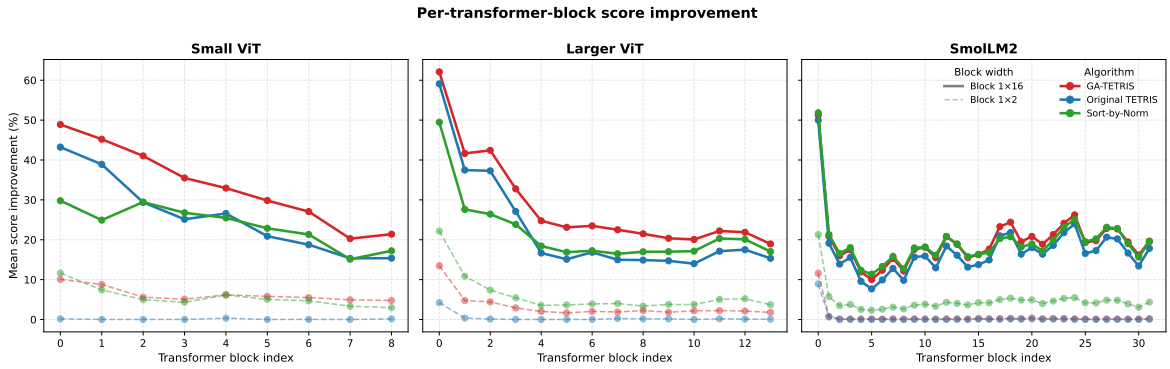

Figure 3.4: Mean (layer-wise) score improvement (%) of selected algorithms across transformer blocks of each model.

Another way to disaggregate the per-layer score improvements is by layer type within a block. The algorithm differences vary considerably across layer types (Figure 3.5). The attention output projection (`attn.proj`) consistently shows the lowest scores on both ViTs, producing roughly half the improvement the algorithms achieve on the other layer types. On SmolLM2, both the attention output projection (`o_proj`) and the gated MLP layers (`mlp.gate_proj`, `mlp.up_proj`) score lowest. We hypothesize that this may be influenced by the fact that the output projections lack the distinct semantic role which QKV or the MLP transformations have. We do not have an analogous explanation for the SmolLM2 gated MLP layers; the poor performance of the algorithms there may simply be a feature of this specific architecture.

Figure 3.5: Mean score improvement (%) of selected algorithms across layer types of each model.

The figures so far report mean score improvement averaged across layers, layer types or transformer blocks. Figure 3.6 shows the per-layer dispersion in the second

metric, importance-weighted reconstruction error (rel_error), with each dot one layer's Δrel_error against the Block-Wanda baseline. Negative values indicate the algorithm achieves lower rel_error than Block-Wanda. At block 1×2 the algorithm differences are small for all three models and the dots cluster tightly near zero. At block 1×16 the differences extend and the algorithm ranking matches what we saw in score improvement, confirming that the two metrics agree despite their different normalization.

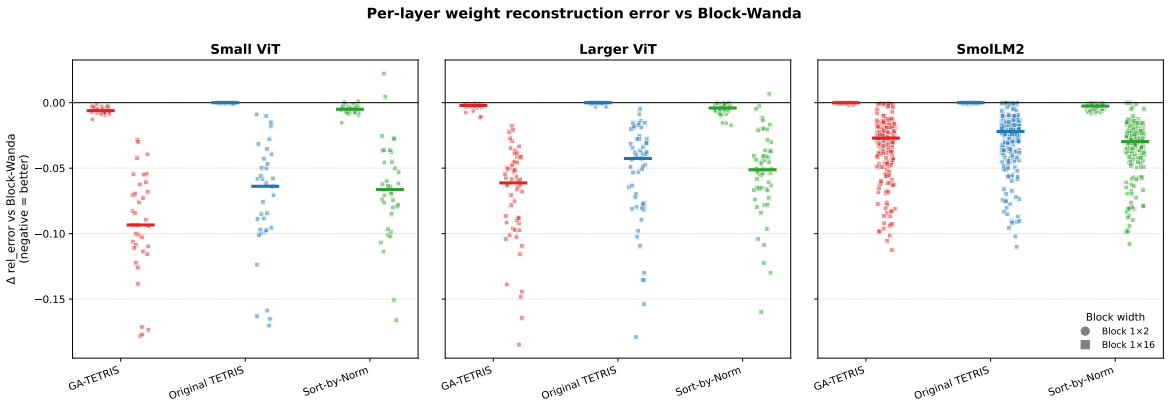

Figure 3.6: Per-layer Δrel_error of selected algorithms against the Block-Wanda baseline. Each dot is one layer; the horizontal bar marks the median.

3.3 Whole model evaluation

Surprisingly, despite the pretty high score improvements compared to the baseline Block-Wanda, the whole-model performance collapses rapidly with the growing block width across all tested models. Figure 3.7 shows the trajectories.

Among our three models, only the Larger ViT preserves substantial accuracy at block 1×2 ($\sim 52-55\%$, against an unpruned baseline of 80%) and partial accuracy at 1×4. The Small ViT and SmolLM2 are unusable at all block widths tested, with accuracy near zero on the Small ViT and perplexity at least in the high tens on SmolLM2.

Within this single setting, GA-TETRIS is the strongest algorithm: 55.11% accuracy versus Block-Wanda's 52.51%, a 2.6 percentage-point gain. The other algorithms only slightly outperform Block-Wanda, most notably, Original TETRIS outperforms it only by 0.13 percentage points. The cost-quality tradeoff is unfavorable in absolute terms — GA-TETRIS spends approximately 7.6 seconds per layer (Table 3.5) for 2.6 percentage points over the no-permutation baseline. Whether this gain is worth the compute depends on the scenario; for a one-time pruning step on a model, it likely is.

On SmolLM2 at block 1×2, the algorithm ranking differs from the Larger ViT result. Sort-by-Norm achieves the lowest perplexity (71.7) with GA-TETRIS substantially

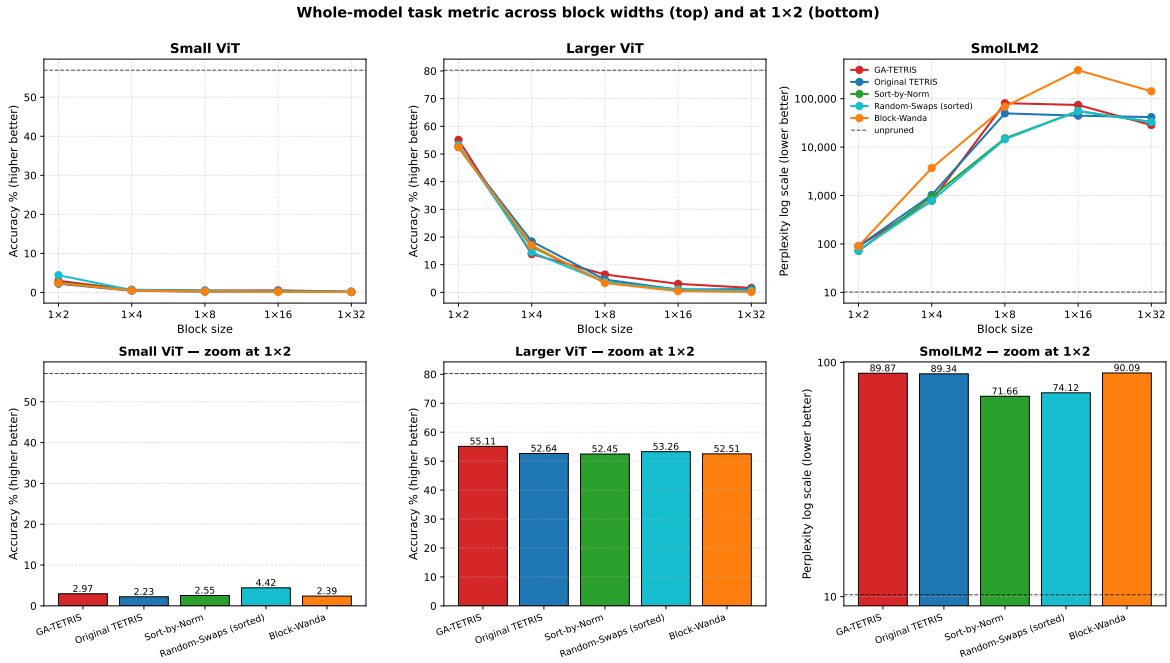

Figure 3.7: Whole-model performance after pruning across block widths and models and zoom on 1×2 block shape.

worse at 89.9. This matches the per-layer score ranking on this regime: the simpler sort-based heuristics win on both metrics. The dominance of Sort-by-Norm here is consistent with our earlier observation that the Wanda-score concentration at narrow block widths leaves little room for combinatorial search to improve the permutation.

Table 3.5: Whole-model performance at block 1×2 on the Larger ViT.

Algorithm	Accuracy	Time per layer (s)	Δ vs GA-TETRIS
GA-TETRIS	**55.11**	7.645	
Random-Swaps (sorted)	53.26	2.234	−1.85 pp
Original TETRIS	52.64	0.012	−2.47 pp
Random-Swaps	52.56	2.301	−2.55 pp
Block-Wanda	52.51	<0.001	−2.60 pp
Sort-by-Norm	52.45	0.002	−2.66 pp
Unpruned baseline	80.27		

We have to note here that GA-TETRIS has reduced swap budget ($T_{phase2} = 10$, Section 3.1) compared to the smaller models. To test the effect of this reduction, we re-ran GA-TETRIS with increased swap budget $T_{phase2} = 100$ on SmolLM2 at just this block width. This increase in swap budget improved the perplexity to 85.7, a 4 percentage-point gain over the $T_{phase2} = 10$ run, but still substantially worse than

Sort-by-Norm's 71.7. The cost of this improvement is also high: the time per layer increases from 7.645 seconds to 20.9 seconds. Even at this much higher budget, GA-TETRIS remains substantially worse than Sort-by-Norm which on top of this has a much smaller time requirements of 0.002 seconds per layer.

Conclusion

This thesis introduced *Global Assignment TETRIS* (GA-TETRIS), a permutation search algorithm for block pruning of transformer weight matrices. The central idea is that finding the best permutation at a fixed block mask is exactly the linear assignment problem and can be solved in polynomial time, where the Original TETRIS uses a greedy single-swap heuristic that stops at the first local optimum. To this exact step we added two heuristics — multiplicative noise injection during the assignment phase and a second phase of random-swap-with-refinement — to escape local optima created by the iterative re-masking. We also introduced two simple heuristics, Sort-by-Norm and Random-Swaps, to compare against the more complex TETRIS algorithms.

We compared GA-TETRIS against the Original TETRIS, Sort-by-Norm, Random-Swaps baselines, and the no-permutation Block-Wanda baseline on two Vision Transformers (0.9M and 13.4M parameters) and SmolLM2-360M. The per-layer evaluation showed that GA-TETRIS produces the largest score improvements at wide block widths consistently across all three models, with the gains concentrated in the early transformer blocks. The crossover block width at which GA-TETRIS overtakes simple Sort-by-Norm was approximately the same across models, suggesting it is a property of the Wanda-score distribution rather than of model size.

The whole-model picture was less favorable. Performance collapsed sharply with block width on all three models, with only the Larger ViT at block 1×2 preserving substantial accuracy. In that single regime GA-TETRIS achieved the highest accuracy, beating the next-best algorithm by 1.85 percentage points and the no-permutation baseline by 2.6 percentage points. On SmolLM2 at the same block width, however, surprisingly the simple Sort-by-Norm produced the lowest perplexity — though still roughly seven times worse than the unpruned baseline — with both Original TETRIS and GA-TETRIS noticeably worse. The compute cost of GA-TETRIS is also substantial: at block 1×2 on the Larger ViT it ran approximately 7.6 seconds per prunable layer, against essentially zero for Sort-by-Norm. Whether the additional accuracy is worth the compute depends on the deployment context.

As future work, several directions remain open. We did not fine-tune the pruned models, which is standard practice in the pruning literature and might substantially recover task quality. We also operated at reduced compute budgets in places: GA-

TETRIS used $T_{phase2} = 100$, and Random-Swaps used $T_{swaps} = 1000$. We saw a good evidence that the algorithms didn't converge at the given random swap limits and could potentially benefit from more iterations at the cost of multiple hours compute time on larger models. Apart from this, the sharp gap between per-layer score improvement and whole-model task quality, especially on SmolLM2, suggests that the entrywise L_1 pruning objective is not perfectly aligned with what determines downstream behavior. Finally, the algorithm has hyperparameters we did not tune systematically, in particular the swap fraction ρ and the initial noise scale η_0.

Bibliography

[1] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate, 2016.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.

[4] Chandra R. Chegireddy and Horst W. Hamacher. Algorithms for finding k-best perfect matchings. *Discrete Applied Mathematics*, 18(2):155–165, 1987.

[5] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling, 2014.

[6] David Crouse. On implementing 2d rectangular assignment algorithms. *IEEE Transactions on Aerospace and Electronic Systems*, 52:1679–1696, 08 2016.

[7] Timoth'ee Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. Vision transformers need registers. *arXiv preprint arXiv:2309.16588*, 2023.

[8] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022.

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

[10] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*, 2021.

[11] Jack Edmonds and Richard M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *J. ACM*, 19(2):248–264, April 1972.

[12] François Fleuret. Deep Learning Course, Lecture 13.2: Attention Mechanisms. Course slides, University of Geneva (UNIGE 14x050), 2024. Accessed: *2026-03-17*.

[13] Yuan Gong, Yu-An Chung, and James Glass. Ast: Audio spectrogram transformer, 04 2021.

[14] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. `http://www.deeplearningbook.org`.

[15] Torsten Hoefler, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks, 2021.

[16] Y. Ji, L. Liang, L. Deng, Y. Zhang, Y. Zhang, and Y. Xie. TETRIS: tile-matching the tremendous irregular sparsity. *Advances in Neural Information Processing Systems*, pages 4119–4129, 2018.

[17] Tommie Kerssies, Niccolò Cavagnero, Alexander Hermans, Narges Norouzi, Giuseppe Averta, Bastian Leibe, Gijs Dubbelman, and Daan de Geus. Your vit is secretly an image segmentation model, 2025.

[18] Harold W. Kuhn. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, March 1955.

[19] Yanghao Li, Hanzi Mao, Ross Girshick, and Kaiming He. Exploring plain vision transformer backbones for object detection, 2022.

[20] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

[21] Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks, 2021.

[22] Sharan Narang, Eric Undersander, and Gregory Diamos. Block-sparse recurrent neural networks, 2017.

[23] Narges Norouzi, Idil Esen Zulfikar, Niccolò Cavagnero, Tommie Kerssies, Bastian Leibe, Gijs Dubbelman, and Daan de Geus. Videomt: Your vit is secretly also a video segmentation model, 2026.

[24] Alec Radford and Karthik Narasimhan. Improving language understanding by generative pre-training. 2018.

[25] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023.

[26] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015.

[27] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models, 2024.

[28] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks, 2014.

[29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[30] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.

[31] Ross Wightman. Pytorch image models. `https://github.com/huggingface/pytorch-image-models`, 2019.

Appendix A: Electronic attachment

The electronic attachment contains the code used in this thesis: the implementation of the pruning algorithms, the Jupyter notebooks for running the pruning and evaluation on the models, the scripts for running the experiments, and the CSV results together with the Jupyter notebooks used to generate the figures used in this thesis. A README file with instructions for setting up and running the code is included. The code is also available on GitHub at `https://github.com/jitka1997/master-thesis/tree/main/code`.